MedWatch — Backend Feature Test Plan: Explanation for Manager

Concise descriptions of the 17 Postman test points and what they do in the project

This document explains each Postman test step and why it matters for MedWatch. Use this during demos or as talking points for your boss.

1. Health check

A simple GET to '/' to confirm the backend server is running and reachable. Useful as a first-step smoke test before any other calls.

2. Create first admin (register)

Registers the first administrative user. If the database has no users, this creates an initial 'admin' account used to manage the system (create doctors, staff, configure map layout).

3. Login (get token)

Authenticates a user (admin/doctor/staff) and returns a JWT token. The token is required to call protected endpoints and to authenticate socket connections for realtime updates.

4. Create non-admin user (doctor/staff)

Register additional users with roles such as 'doctor' or 'staff'. Roles control access to sensitive operations and UI features.

5. Ingest sheet rows

Endpoint that receives rows from Google Sheets (Apps Script) or Postman and writes raw RFID events into the database (raw_events). This is the primary data ingestion path.

6. Compute contact edges

Processes raw_events and creates contact edges (contact_edges) by detecting overlapping presence in the same room/time. This builds the contact graph used for tracing.

7. Check contact chain (BFS graph)

Returns a BFS-limited subgraph (nodes & edges) for a given UID up to a requested depth (primary, secondary, tertiary). Useful for visualizing exposure chains.

8. Get live map (last-known + risk)

Provides each UID's last known room and computed risk status (primary/secondary/tertiary/safe). The frontend uses this to plot the interactive hospital map with colored statuses.

9. Create/Update map layout (Admin)

Admin endpoint to store room coordinates and metadata (x,y,width,height,floor,risk_zone). The frontend uses layout data to render a visual map overlay.

10. Get map layout

Returns stored room layout data for the frontend so it can plot rooms and position people correctly on the interactive map.

11. Patient search (tracing)

Search endpoint to find patients/staff by name or UID. Useful for starting contact-tracing investigations or pulling up patient history quickly.

12. Patient detail (history)

Returns a patient's profile and recent RFID events (movement history). Helps investigators review timeline and exposures.

13. Mark an MDR case — create alerts

Marks a UID as MDR-positive (mdr_cases) and automatically generates Alerts for primary, secondary, and tertiary contacts. Emits realtime notifications to teams and affected users.

14. Query alerts (all or by target)

Allows viewing stored alerts, optionally filtered by a target UID. Used by staff to see who must be tested, isolated, or monitored.

15. Contacts compute for a single room

Run contact computation for a single room (useful for targeted recalculation after a localized ingest or manual correction).

16. Extra debug queries (DB checks)

Direct SQL queries to inspect raw_events, contact_edges, alerts, and mdr_cases — useful for troubleshooting and verification during demos.

17. Socket.IO quick test (realtime)

Explains how Socket.IO connections are used: clients authenticate with JWT, join UID rooms, and receive real-time events like 'rawevent:new', 'map:update', 'alert:new', and 'contacts:computed'. Useful for live demos of the map and alerts.